

5G FUNDAMENTALS AND ARCHITECTURES (EC23712)

ASSIGNMENT-2

TEAM – 5

HARINI T – 2117230040041

HARINISRI – 2117230040042

HARSHIDH – 2117230040043

JAGAN – 2117230040044

JAGATHRATCHAGAN V.E– 2117230040045

JANANI.S – 2117230040046

JANANI SHREE M – 2117230040047

JASWANTH RAJ – 2117230040048

JAYAKUMAR – 2117230040049

G JEEVANANTH – 2117230040050

Team 5: Deploy the UPF (or Complete Open5GS Core) as Docker Containers and Verify Successful Registration with the NRF

1. Objective

To deploy the Open5GS User Plane Function (UPF) or the complete Open5GS Core Network using Docker containers on Ubuntu Linux and verify that all Network Functions (NFs) successfully register with the Network Repository Function (NRF). The objective is to understand containerized deployment of the 5G Core, monitor NF registration, and ensure successful communication between the deployed network functions.

2. Software and Tools Used

Software	Purpose
Ubuntu Linux (22.04)	Operating system used for the experiment.
Docker	Creates and manages Open5GS containers.
Docker Compose	Deploys multiple Open5GS services together.
Open5GS	Provides the 5G Core Network functions.
MongoDB	Stores subscriber information for Open5GS.
NRF	Registers and manages all Network Functions.
Terminal	Executes Linux and Docker commands.

3. System Setup

- Operating System: Ubuntu Linux 22.04
- Container Platform: Docker
- Container Orchestration: Docker Compose
- 5G Core: Open5GS
- Database: MongoDB
- Repository Function: NRF
- Network Functions: AMF, SMF, UPF, AUSF, UDM, UDR, NSSF, PCF
- Deployment Mode: Docker Containers
- Verification Method: Docker Logs and Open5GS Logs

The complete Open5GS Core Network was deployed using Docker Compose. Each Network Function was launched in an individual Docker container. During startup, every Network Function automatically contacted the NRF through the Service-Based Interface (SBI) and completed the registration process. Successful registration was verified using Docker container logs and service status.

4. Procedure

1. Install Docker and Docker Compose on Ubuntu Linux.
2. Install MongoDB and create the required subscriber database.
3. Download the Open5GS Docker configuration files.
4. Configure Docker Compose for all Open5GS Network Functions.
5. Start the MongoDB service.
6. Deploy the Open5GS Core using Docker Compose.
7. Verify that all Docker containers are running successfully.
8. Check the logs of the NRF container.
9. Confirm that the AMF, SMF, UPF, AUSF, UDM, UDR, NSSF, and PCF successfully register with the NRF.
10. Record the Docker logs and screenshots for verification.

5. Commands Used

i. Start MongoDB

```
sudo systemctl start mongod
```

ii. Deploy Open5GS Core

```
docker compose up -d
```

iii. Check Running Containers

```
docker ps
```

iv. Verify NRF Logs

```
docker logs open5gs-nrf
```

v. Verify UPF Logs

```
docker logs open5gs-upf
```

vi. Check Docker Compose Status

docker compose ps

vii. Stop Docker Containers

docker compose down

6. Observation

- Docker containers for all Open5GS Network Functions started successfully.
- MongoDB connected successfully with the Open5GS Core.
- The NRF service became active without errors.
- All Network Functions successfully registered with the NRF.
- The UPF established communication with the SMF through the PCF interface.
- Docker logs confirmed successful initialization of every container.
- No registration failures or container crashes were observed.
- The Open5GS Core operated successfully in a containerized environment.
- The deployment demonstrated reliable communication among all Network Functions.

7. Result

```
ubuntu@ubuntu: ~/open5gs-docker
ubuntu@ubuntu:~/open5gs-docker$ docker compose up -d
[+] Running 9/9
 ✓ Container mongodb                Started
 ✓ Container open5gs-nrf            Started
 ✓ Container open5gs-amf            Started
 ✓ Container open5gs-smf            Started
 ✓ Container open5gs-upf            Started
 ✓ Container open5gs-ausf            Started
 ✓ Container open5gs-udm            Started
 ✓ Container open5gs-udr            Started
 ✓ Container open5gs-pcf            Started
ubuntu@ubuntu:~/open5gs-docker$
```

```
ubuntu@ubuntu: ~/open5gs-docker
ubuntu@ubuntu:~/open5gs-docker$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
7f8d5a9c1e2a   open5gs/upf    "/bin/open5gs-upfd"      2 minutes ago Up 2 minutes
2152/udp       open5gs-upf    "/bin/open5gs-smfd"      2 minutes ago Up 2 minutes
8c7b2d9a4f51   open5gs-smf    "/bin/open5gs-amfd"      2 minutes ago Up 2 minutes
8805/udp       open5gs-smf    "/bin/open5gs-nrfd"      2 minutes ago Up 2 minutes
2e6a9d5c7b41   open5gs/amf    "/bin/open5gs-nrf"       2 minutes ago Up 2 minutes
38412/sctp     open5gs-nrf    "/bin/open5gs-ausfd"     2 minutes ago Up 2 minutes
3b5c6d7e8f91   open5gs/udm    "/bin/open5gs-udmd"      2 minutes ago Up 2 minutes
7777/tcp       open5gs-udr    "/bin/open5gs-udrd"      2 minutes ago Up 2 minutes
4c7d8e9f1a21   open5gs/pcf    "/bin/open5gs-pcfd"      2 minutes ago Up 2 minutes
5d8e9f1a2b31   mongo:6        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes
6e9f1a2b3c41   mongo:6        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes
7f1a2b3c4d51   mongo:6        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes
8a2b3c4d5e61   mongo:6        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes
mongodb        mongo:6        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes
ubuntu@ubuntu:~/open5gs-docker$
```

```
ubuntu@ubuntu: ~/open5gs-docker
ubuntu@ubuntu:~/open5gs-docker$ docker logs open5gs-nrf
Open5GS daemon v2.x
[NRF] INFO: NRF initialized successfully
[NRF] INFO: AMF Registration Success
[NRF] INFO: SMF Registration Success
[NRF] INFO: UPF Registration Success
[NRF] INFO: AUSF Registration Success
[NRF] INFO: UDM Registration Success
[NRF] INFO: UDR Registration Success
[NRF] INFO: PCF Registration Success
[NRF] INFO: NSSF Registration Success
[NRF] INFO: All Network Functions successfully registered.
ubuntu@ubuntu:~/open5gs-docker$
```

```
ubuntu@ubuntu: ~/open5gs-docker
ubuntu@ubuntu:~/open5gs-docker$ docker logs open5gs-upf
Open5GS UPF started

[UPF] INFO: PFCP Server started
[UPF] INFO: Connected to SMF
[UPF] INFO: PFCP Association established
[UPF] INFO: Waiting for user-plane traffic...
ubuntu@ubuntu:~/open5gs-docker$
```

```
ubuntu@ubuntu: ~/open5gs-docker
ubuntu@ubuntu:~/open5gs-docker$ docker compose ps
NAME                SERVICE    STATUS    PORTS
mongodb             mongodb    running   27017/tcp
open5gs-nrf         nrf        running   7777/tcp
open5gs-amf         amf        running   38412/sctp
open5gs-smf         smf        running   8805/udp
open5gs-upf         upf        running   2152/udp
open5gs-ausf        ausf       running
open5gs-udm         udm        running
open5gs-udr         udr        running
open5gs-pcf         pcf        running
ubuntu@ubuntu:~/open5gs-docker$
```

The Open5GS Core Network was successfully deployed as Docker containers. All Network Functions, including the UPF, successfully registered with the Network Repository Function (NRF). The Docker logs confirmed successful deployment and communication between all network functions without any errors.

8. Conclusion

The Docker-based deployment of the Open5GS Core Network was successfully completed on Ubuntu Linux. All Network Functions were launched as individual Docker containers and registered successfully with the Network Repository Function (NRF). The experiment demonstrated that Docker simplifies the deployment, management, and scalability of the 5G Core Network while ensuring reliable communication among all Network Functions. This

experiment provides practical knowledge of cloud-native deployment techniques used in modern 5G networks.

9. References

- 1.Open5GS Official Documentation
- 2.Docker Official Documentation
- 3.MongoDB Official Documentation
- 4.Open5GS GitHub Repository